

sum's blog

Editorial for Codeforces Round #919 (Div. 2)

 By [sum](#), [history](#), 3 years ago, 

Information about the round

[Rating predictions \(inspired by BucketPotato's editorial\)](#)
[Who did what](#)

Solutions

1920A - Satisfying Constraints

[Hint 1](#)
[Solution](#)
[Code](#)

1920B - Summation Game

[Hint 1](#)
[Hint 2](#)
[Solution](#)
[Code](#)

1920C - Partitioning the Array

[Hint 1](#)

Try to solve the problem for just two integers x and y . Under what m are they equal (modulo m)?

[Hint 2](#)

How can we use the previous hint and gcd to solve the problem?

[Solution](#)

For some x and y , let's try to find all m such that $x \bmod m \equiv y \bmod m$. We can rearrange the equation into $(x - y) \equiv 0 \pmod{m}$. Thus, if m is a factor of $|x - y|$, then x and y will be equal modulo m .

Let's solve for some k . A valid partition exists if there exists some $m > 1$ such that the following is true:

- $a_1 \equiv a_{1+k} \pmod{m}$
- $a_2 \equiv a_{2+k} \pmod{m}$
- ...
- $a_{n-k} \equiv a_n \pmod{m}$

The first condition $a_1 \equiv a_{1+k} \pmod{m}$ is satisfied if m is a factor of $|a_1 - a_{1+k}|$. The second condition $a_2 \equiv a_{2+k} \pmod{m}$ is satisfied if m is a factor of $|a_2 - a_{2+k}|$. And so on...

Thus, all conditions are satisfied if m is a factor of:

→ Pay attention

Before contest

[Codeforces Round 1109 \(Div. 3\)](#)

36:08:43

[Register now »](#)

→ jiashenli40



Rating: 434

Contribution: 0

- [Settings](#)
- [Blog](#)
- [Teams](#)
- [Submissions](#)
- [Favourites](#)
- [Talks](#)
- [Contests](#)



jiashenli40

→ Top rated

#	User	Rating
1	jiangly	3728
2	VivaciousAubergine	3647
3	Kevin114514	3625
4	maroonrk	3555
5	strapple	3515
6	tourist	3439
7	Um_nik	3376
7	turmax	3376
9	Radewoosh	3367
10	heuristica	3322

[Countries](#) | [Cities](#) | [Organizations](#)
[View all →](#)

→ Top contributors

#	User	Contrib.
1	Qingyu	163
2	Um_nik	145
3	adamant	142
4	Dominator069	136
5	chromate00	133
5	DNR	133
5	maroonrk	133
8	cry	132
8	maspy	132
10	Proof_by_QED	131

[View all →](#)



$$|a_1 - a_{1+k}|, |a_2 - a_{2+k}|, \dots, |a_{n-k} - a_n|$$

In other words, all conditions are satisfied if m is a factor of:

$$\gcd(|a_1 - a_{1+k}|, |a_2 - a_{2+k}|, \dots, |a_{n-k} - a_n|)$$

So a valid m exists for some k if the aforementioned gcd is greater than 1.

We can iterate over all possible k (remember that k is a divisor of n) and solve for each k to get our answer. The time complexity of this will be $O((n + \log n) \cdot \max \text{divisors of } n)$.

Note that each pass through the array takes $n + \log n$ time because of how the gcd will either be halved or stay the same at each point.

Code

1920D - Array Repetition

Hint 1

Hint 2

Solution

Code (iterating over second type operations)

Code (repeated binary searches)

1920E - Counting Binary Strings

Hint 1

Hint 2

Hint 3

Solution

Code

1920F1 - Smooth Sailing (Easy Version)

Hint 1

Hint 2

Solution

Code

1920F2 - Smooth Sailing (Hard Version)

Hint 1

Hint 2

Hint 3

Solution

Code (small to large)

Code (LCA queries)

Tutorial of Codeforces Round 919 (Div. 2)

+314

sum

3 years ago

183



Comments (121)

Show archived | Write comment?



knoothopian

2 years ago, hide # | ☆

Good round. Had fun solving A, B, C. Kudos for fast editorial!

→ Reply

+3

Find user

Handle:

Find

Recent actions

atcoder_official → [AtCoder Regular Contest-- 224 Announcement](#)

[_baozii](#) → [Spectral::Cup 2026 Round 3 \(Codeforces Round 1110, Div. 1 + Div. 2\)](#)

[ezraft](#) → [Codeforces Round 1108 \(Div. 2\)](#)

[Ahmad_OS](#) → [How do you debug?](#)

[DNR](#) → [AI resolves yet another major conjecture in math](#)

[Omar](#) → [Codeforces Round 1108 \(Div. 2\) Editorial](#)

[itz_pabloo](#) → [Codeforces Round 1109 \(Div. 3\)](#)

[abhi_yo](#) → [Grid problem](#)

[MikeMirzayanov](#) → [Rule Restricting the Use of AI \[revision 2024-09-14\]](#)

[VanLam](#) → [Solution Contest đầu vào CLB IT PTIT 2025](#)

[dino_merlin](#) → [Finally specialist again after 3 long, horrible, exruciating years](#)

[accord](#) → [Competitive Programming in the Era of AI](#)

[BledDest](#) → [Educational Codeforces Round 192 Editorial](#)

[makeinverse](#) → [CP Insights — Beautiful Codeforces Cards for GitHub READMEs](#)

[AmShZ](#) → [Repovive Premier Round 6](#)

[mambo](#) → [how are we not all suicidal](#)

[pushedx](#) → [How do I set my home country on Codeforces?](#)

[ShulkerBox](#) → [Smallest CF contest ever...](#)

[brave_art_37](#) → [Stuck in CP 1.5 years during college. Pls help](#)

MidnightCodeCup → [Midnight Code Cup Finals 2026: Problems, Winners and More](#)

[sarvan_2187](#) → [Hello, A Bit about Me!](#)

[MikeMirzayanov](#) → [Rule about third-party code is changing](#)

[NetSpeed1](#) → [Codeforces Global Round 28 Editorial](#)

[ABalobanov](#) → [Codeforces Round 1070 \(Div.2\) Editorial](#)

atcoder_official → [AtCoder Beginner Contest 466 Announcement](#)

[Detailed →](#)

**biximo**2 years ago, [hide](#) <#> | ☆ ▲ +16 ▼

Thank you for the great contest, the DP recurrence for E was a bit trivial but OK for Div2E.

→ [Reply](#)**CrypticOwl5503**2 years ago, [hide](#) <#> | ☆ ▲ +1 ▼

Thanks for the fast editorial!

→ [Reply](#)**MaxBlazelcelnk**2 years ago, [hide](#) <#> | ☆ ▲ 0 ▼

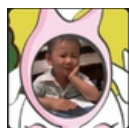
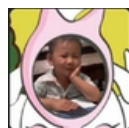
Speed Forces!

→ [Reply](#)**SAYEM_19**2 years ago, [hide](#) <#> | ☆ ▲ +5 ▼

Thanks all for the super fast editorial

→ [Reply](#)**1075508020060209tc**2 years ago, [hide](#) <#> | ☆ ▲ 0 ▼

light speed editorial

→ [Reply](#)**tminh_hk20**2 years ago, [hide](#) <#> | ☆ ▲ 0 ▼Problem C: why we take GCD of all value? we can take $m=2$ to check instead?→ [Reply](#)**omeganot**2 years ago, [hide](#) <#> ^ | ☆ ▲ +33 ▼It is possible that $m=3$ works while $m=2$ doesn't, for example $[2, 3]$, $[5, 6]$ are the same for $m = 3$ but not $m = 2$. So you have to use gcd to find the largest m that works.→ [Reply](#)**tminh_hk20**2 years ago, [hide](#) <#> ^ | ☆ ▲ +5 ▼

Thank!

→ [Reply](#)**KhoaDuy**2 years ago, [hide](#) <#> | ☆ ▲ +1 ▼Can someone explain to me why $O((n+\log n) \cdot \max \text{divisors of } n)$ is fast enough to pass the tests?→ [Reply](#)**omeganot**2 years ago, [hide](#) <#> ^ | ☆ ▲ +26 ▼The sum of n over all tests does not exceed $2 \cdot 10^5$, and the maximum divisors a number less than or equal to $2 \cdot 10^5$ can have is not more than 200 (I think it is around 160 or something). So in the end this is small enough to pass the problem.→ [Reply](#)



Lol2004

2 years ago, [hide](#) <#> [^](#) | [☆](#)

0

You can practically check that for all numbers N not greater 200000, the number of divisors is actually $O(\sqrt[3]{N})$, so the author solution would not take too much operations

→ [Reply](#)

2147483648

2 years ago, [hide](#) <#> [^](#) | [☆](#)

← Rev. 2 0

No, it's $O(e^{\ln(\ln(n))^{1.8}})$.

→ [Reply](#)

Sheyer

2 years ago, [hide](#) <#> [^](#) | [☆](#)

0

How can you prove this? Thank you in advance!

→ [Reply](#)

2147483648

2 years ago, [hide](#) <#> [^](#) | [☆](#)

+3

You can read details here: [link](#)

→ [Reply](#)

r57shell

2 years ago, [hide](#) <#> [^](#) | [☆](#) 0

the link doesn't have a proof

→ [Reply](#)

ANNIHILLATOR

2 years ago, [hide](#) <#> [0](#) [^](#) | [☆](#)

Graph is a proof, lol.
What proof do you want?

→ [Reply](#)

r57shell

2 years ago, [hide](#) <#> [^](#) | [☆](#)

0



$O(f(n))$ has definition.
You should prove that $f(n)$ satisfy the definition.

→ [Reply](#)

second_saturday

2 years ago, [hide](#) <#> [^](#) | [☆](#)

may be this is proof by example, they tested it over large number, found the relation between number of divisors verses N , it roughly approximate



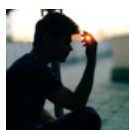
→ [Reply](#)

2 years ago,
[hide](#) #
 ^ | ☆

There is no such thing as a proof by an example. There is only disproof by a counter example. I believe in the case above there is a proof, it's just not what is linked.
 → [Reply](#)



[r57shell](#)



[second_saturday](#)

2 years ago,
[hide](#) #
 ^ | ☆

density of primes for given N is $1/\log(N)$, that relations is also found by only by plotting the graph and there is no exact proof for it...., and we use it very often,

so this relation is also same way found I think...



→ [Reply](#)

2 years ago, [hide](#) # ^ | ☆

There is a proof. Several of them. Google "prime number theorem" for more details. "The elementary proof of the prime number theorem: an historical perspective" (by D. Goldfeld) has elementary proof that $\pi(n)$ is $O(\frac{n}{\log n})$ on pages 1 and 2 which is really really easy, but this statement is not the same as prime number theorem itself.

→ [Reply](#)



r57shell

2 years ago [hide](#) # ^ | ☆

There's no sense of proving it in the context of competitive programming. In every real case the precision of this approximation will be sufficient.

→ [Reply](#)



Markussian



r57shell

2 years ago, [hide](#) # ^ | ☆
without



proof
may be a
lie. $O()$ is
fact, or
not. You
can't say
it's $O()$ if
it's not,
except if
you ok
with a lie.
Regarding
demands
of proof: I
didn't
demand. I
just
wanted to
stress out
that the
page
doesn't
have a
proof. And
claim that
it has a
proof is a
lie.

→ [Reply](#)



a_little_cute

2 years ago, [hide](#) <#> [^](#) | [☆](#)

it is about $200000 \cdot 160 = 3.2 \cdot 10^8$ which can pass in 3 sec

→ [Reply](#)

▲ 0 ▼



yogman

2 years ago, [hide](#) <#> | [☆](#)

Thanks for fast tutorial.

→ [Reply](#)

▲ +8 ▼



ramaaa

2 years ago, [hide](#) <#> | [☆](#)

Wow, Problem C and Problem D are pretty cool. Loved the questions. Thanks
!!!!!!!

→ [Reply](#)

▲ +15 ▼



yogman

2 years ago, [hide](#) <#> [^](#) | [☆](#)

Absolutely Correct!!

→ [Reply](#)

▲ +6 ▼



619

2 years ago, [hide](#) <#> | [☆](#)

Loved task B, Thanks for this high quality round.

→ [Reply](#)

▲ 0 ▼



Brachistochron3

2 years ago, [hide](#) <#> | [☆](#)

if there are more than one occurrence of type 3 constraints with the same x, some
solutions may be hacked!!

→ [Reply](#)

▲ 0 ▼

**femboy-wannabe**2 years ago, [hide](#) # [^](#) | [☆](#)[+8](#)

"no two constraints are the exact same." So there can't be more than one occurrence of the same type with the same x

→ [Reply](#)**femboy-wannabe**2 years ago, [hide](#) # | [☆](#)[+2](#)

Do you expect people to know that the number of divisors of n is small in Div2C? It took me a while to realize that you can just check each divisor in $O(n)$, I bet there were a lot of people who just submitted it without understanding why it works fast

→ [Reply](#)**awesomguy856**2 years ago, [hide](#) # [^](#) | [☆](#)[+69](#)

It's a pretty common idea in problems and well known enough

I think most people at least know that the number of divisors is at most $2\sqrt{n}$, and even if you use this extreme bound, $O(n \sqrt{n})$ still passes under the constraints

→ [Reply](#)**femboy-wannabe**2 years ago, [hide](#) # [^](#) | [☆](#)[+17](#)

Oh, forgot about $2\sqrt{n}$ bound, yes that makes much more sense... I'm now even more ashamed of spending so much time on this problem

→ [Reply](#)**biximo**2 years ago, [hide](#) # [^](#) | [☆](#)[+14](#)

For future reference, most of the time it's ok to assume max # of divisors is around $N^{1/3}$

→ [Reply](#)**femboy-wannabe**2 years ago, [hide](#) # [^](#) | [☆](#)[+3](#)

Thanks! I already knew that but apparently not well enough..

→ [Reply](#)**nathanballman**2 years ago, [hide](#) # [^](#) | [☆](#) Rev. 2[0](#)

You can get the exact bound by finding the number of divisors of the greatest highly composite number less than the constraint. You can find these numbers in this series on oeis <https://oeis.org/A002182>. These could also be useful when writing tests/hacks for problems.

→ [Reply](#)**chai.pi**2 years ago, [hide](#) # [^](#) | [☆](#)[0](#)

Okay, I'll bite. how cube root?

→ [Reply](#)**BananaBrain**2 years ago, [hide](#) # [^](#) | [☆](#)[0](#)

What is this $2\sqrt{n}$ bound about, I am still confused

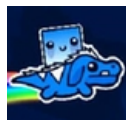
→ [Reply](#)



leessjn

2 years ago, [hide](#) <#> [^](#) | [☆](#)[▲](#) 0 [▼](#)

So I added other constraints $\text{ars}[i-1] \cdot (b+1) < \text{ars}[i-1]$ too.. Can that also lead to overflow..?

→ [Reply](#)

1xx55

2 years ago, [hide](#) <#> [^](#) | [☆](#)[▲](#) 0 [▼](#)

I think $\text{ars}[i-1] \leq 1e18 / (b+1)$ is better than $\text{ars}[i-1] \cdot (b+1) < \text{ars}[i-1]$. When you calculate $\text{ars}[i-1] \cdot (b+1)$ then it already overflow at some cases and it is possible that $\text{ars}[i-1] \cdot (b+1) \geq \text{ars}[i-1]$

→ [Reply](#)

Sheyer

2 years ago, [hide](#) <#> [^](#) | [☆](#)[▲](#) 0 [▼](#)

Yes, it may in some cases like in my submission: [241476717](#). you better use $1e18 / (b+1) \geq \text{ars}[i-1]$. I still don't know why $\text{ars}[i-1] \cdot (b+1) < \text{ars}[i-1]$ can overflow, any explanation is appreciated!

→ [Reply](#)2 years ago, [hide](#) <#> [^](#) | [☆](#)[▲](#) +3 [▼](#)

Try to run this code:

```
long long a =
922337203685477632;
cout << (a * 21);
```

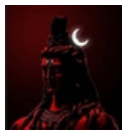


r57shell

It overflows, $a < 1e18$ and result $> a$. Basic way to find out **a** similar to this is to solve:

$x \cdot k = (2^{64}) + x$ Solution is: $\frac{2^{64}}{k-1}$ I picked $k = 21$ and python with its precision errors:

```
>>> int((2**64)/20)
922337203685477632
→ Reply
```



suyasho786

2 years ago, [hide](#) <#> | [☆](#)[▲](#) +1 [▼](#)

Fast Editorials :)

→ [Reply](#)

maxrgby

2 years ago, [hide](#) <#> | [☆](#)← Rev. 2 [▲](#) 0 [▼](#)

Here's an alternate solution to problem C without GCD. Observe that if we need to pick some m such that a set of numbers are all congruent to each other modulo m , m is at most as large as the second smallest element in the set. The bound on m is small enough to bruteforce all k and all m and it somehow works.

Code: [241481077](#)→ [Reply](#)

Nachia

2 years ago, [hide](#) <#> [^](#) | [☆](#)[▲](#) +6 [▼](#)

Excuse me but somehow your submission may TLE when the input is like :

```
1
200000
30030 30030 30030 ... 30030 1
```



if from submit since its the same as the hack input for `12 1 1000000000` ,
→ [Reply](#)



maxrgby

2 years ago, [hide](#) <#> [^](#) | ☆

▲ +3 ▼

Thanks, good to know! Actually, my solution did get hacked.
Seems like the system tests were weak.

→ [Reply](#)



IgrisTheRed

2 years ago, [hide](#) <#> [^](#) | ☆

▲ 0 ▼

I'm not able to observe. Could you please elaborate a bit more on why this works?

→ [Reply](#)



aryanc403

2 years ago, [hide](#) <#> [^](#) | ☆

▲ +7 ▼

Someone send me this solution link in my [live stream](#) solution discussion.

I hacked it live there :)

The hack part starts at [1:01:16](#)

Since, you are iterating on all nos until upper, I chose `thing = 2*p` and `upper = p` , where p is a prime.

→ [Reply](#)



flakes24

2 years ago, [hide](#) <#> | ☆

← Rev. 3 ▲ 0 ▼

was WA on pretest 3 on D meant to catch any known error or just a random case? :(not able to find my mistake

→ [Reply](#)



Adityavb05

2 years ago, [hide](#) <#> | ☆

← Rev. 4 ▲ +5 ▼

Alternate Solution to problem C.

For a given divisor of N, its sufficient enough to check if there exists any prime which satisfies the given conditions (Not so hard to prove why it is so) . Since there are not many primes under $2.1e5$ the solution easily passes for the given constraints . Submission [241489844](#). Edit: Lmao TL hacked

→ [Reply](#)



gurmeet_singh2003

2 years ago, [hide](#) <#> | ☆

▲ +1 ▼

I have a question how problem C $O(n^2)$ solution is being getting accepted ?

→ [Reply](#)



_Spongey

2 years ago, [hide](#) <#> [^](#) | ☆

▲ 0 ▼

I think you mean $O(n \cdot (2\sqrt{n}))$? if you're talking about the gcd.

→ [Reply](#)



gurmeet_singh2003

2 years ago, [hide](#) <#> [^](#) | ☆

▲ 0 ▼

No, I have seen many solutions which are $O(n^2)$ as well. They are getting accepted. How ? They are directly running for loop for findings divisors as well.

→ [Reply](#)



vgtcross

2 years ago, [hide](#) <#> [^](#) | ☆

▲ 0 ▼

Can you link one such $O(n^2)$ solution which got accepted?

→ [Reply](#)



2 years ago, [hide](#) # [^](#) | [☆](#) [▲](#) 0 [▼](#)

Sure, Here is that: <https://codeforces.com/contest/1920/submission/241451239>

→ [Reply](#)



apollo301269

2 years ago, [hide](#) # [^](#) | [☆](#) [▲](#) 0 [▼](#)

There is a check

`if(n%k==0)` inside the loop, so the complexity is the same as in the tutorial.

→ [Reply](#)

2 years ago, [hide](#) # [^](#) | [☆](#) [▲](#) +13 [▼](#)

This is not $\Theta(n^2)$.

The inner loop runs only if k is a factor of n , i.e. $d(n)$ times, where $d(n)$ is the number of factors of n .

The inner loop does $n - k \in O(n)$ iterations; each iteration calls `gcd` once which seems like it would be $n \log \max a$ in total, but since we calculate gcd with the previous result, the total runtime is $n + \log \max a$ (check [this](#)).

Since the above is done $d(n)$ times, the time complexity is $O((n + \log \max a)d(n))$. For $n \leq 2 \cdot 10^5$, $d(n)$ is at most 160 (this is reached for $n = 196560$; you can verify it yourself if you wish). This is fast enough.

→ [Reply](#)



KAZHYI

2 years ago, [hide](#) # [^](#) | [☆](#)

I can't justify my D-time complexity. [this is my D](#)

→ [Reply](#)



_Spongey

2 years ago, [hide](#) # [^](#) | [☆](#)

solution for problem D**

→ [Reply](#)

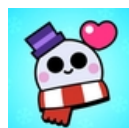


KAZHYI

2 years ago, [hide](#) # [^](#) | [☆](#)

<https://codeforces.com/contest/1920/submission/241488534>

→ [Reply](#)



_White_Knight_

2 years ago, [hide](#) # [^](#) | [☆](#)

<https://codeforces.com/contest/1920/submission/241478736> what is wrong in this code for problem C?

→ [Reply](#)

2 years ago, [hide](#) <#> | ☆

▲ +6 ▼

Here is an alternative solution for F1 (in practice: [241489058](#)).

First, run a multi-source BFS from all volcanoes to determine the distance from every square to the nearest volcano.

Now, given a query, we run another kind of BFS from the given square. Let d be the distance from this square to the nearest volcano. Instead of one queue, we maintain $d + 1$ queues: the squares reachable when the minimum distance to the volcanoes is $0, 1, 2, \dots, d$. Initially, the d -th queue contains our starting square, and we process the queues in the order from highest to lowest.



Gassa

The remaining issue is to detect when we made a turn around the island. For that, pick an arbitrary square of the island in advance. For each visited square, maintain a real value: how the polar angle changed when getting to that square. When the square was not yet visited, just record the current value. When we arrive at an already visited square: if the difference between the old and the new value is close to zero, do nothing; but if it is not (most probably close to $\pm 2\pi$ then), we just found a round trip. What remains is to record the greatest possible distance from volcanoes when this happened.

→ [Reply](#)

Christiano_7

2 years ago, [hide](#) <#> | ☆

▲ 0 ▼

Can someone please explain in problem B why $n \log n$ is passing? Because number of testcases are $1e4$ and n is $1e5$ and $k \leq n$ so when $k=1$ and $n=1e5$ and $t=1e4$ will the $n \log n$ solution will be accepted?

→ [Reply](#)

vgtcross

2 years ago, [hide](#) <#> | ☆

▲ 0 ▼

It is guaranteed that the sum of n over all test cases does not exceed $2 \cdot 10^5$.

→ [Reply](#)

beyondpluto

2 years ago, [hide](#) <#> | ☆

▲ +2 ▼

Am I the only one who found majority of the problems in this contest to be filler?

→ [Reply](#)

kto_eto

2 years ago, [hide](#) <#> | ☆

← Rev. 2

▲ +11 ▼

F2 can be solved in $O(nm \cdot \alpha(nm) + q)$ with offline LCA.

→ [Reply](#)2 years ago, [hide](#) <#> | ☆

▲ +18 ▼

Hey,

I think the data for Problem C might have been a little weak. The following testcase:



socho

1

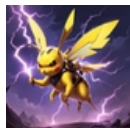
100000

99997 99997 99997 49998

Can hack solutions which use the heuristic that the GCD should be $= 1$ (if more than 1 then that GCD value itself works), and also that the only valid answer is 49999 (which is a large prime). Some solutions try all numbers from $2 \dots 10^5$ and that should usually be too slow, I think.

Thanks!

→ [Reply](#)



madhavG

2 years ago, [hide](#) <#> [^](#) | [☆](#)[▲](#) -8 [▼](#)

precompute divisors SMH

→ [Reply](#)

KisuraOP

2 years ago, [hide](#) <#> | [☆](#)[▲](#) 0 [▼](#)

I personally think C is harder than D. Maybe swap them is better?

→ [Reply](#)

masonpop

2 years ago, [hide](#) <#> | [☆](#)[▲](#) +3 [▼](#)

I misunderstood E's "substring" as "subsequence" and wrote a wrong DP. Now feeling I'm a noob :(

→ [Reply](#)

_Spongey

2 years ago, [hide](#) <#> [^](#) | [☆](#)[▲](#) +4 [▼](#)

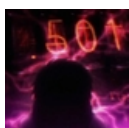
it happens (I'm actually a noob)

→ [Reply](#)2 years ago, [hide](#) <#> | [☆](#)[▲](#) 0 [▼](#)For problem C: I Need some help to figure out what the complexity of this solution is: [241500814](#)It might be around $O(N \cdot \log(N))$ I guess, but I am not sure.

Zixel

So, what I did is to first arrange all factors of n in a 2D vector (let's say V) s.t.in any vector if the elements are like $\{a, b, c, \dots, x, y\}$ then it holds that $b \% a = 0$, $c \% b = 0, \dots, y \% x = 0$.Once we have such arrays we can binary search on each of them to find out for how many of the factors in an array $m > 1$ exists.So if the size of this 2D vector is X , then the complexity would be something like: $O(N \cdot \log(N) \cdot X)$, but what will be the upper bound on X ??→ [Reply](#)

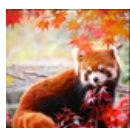
34z12000

2 years ago, [hide](#) <#> [^](#) | [☆](#)← Rev. 2 [▲](#) 0 [▼](#) $n^{1/3}$ → [Reply](#)2 years ago, [hide](#) <#> | [☆](#)← Rev. 3 [▲](#) 0 [▼](#)

Carousel

This was a good round, but the only issue I had was that D had repeated occurrences of indices. I know it was never stated in the input that the indices had to be distinct, but I felt like it was obvious enough to assume haha. Well, at least I learned to be a lot more careful.

I do think that at least one of the examples should have featured repeated indices though.

→ [Reply](#)

Dominater069

2 years ago, [hide](#) <#> [^](#) | [☆](#)[▲](#) -14 [▼](#)

you cant expect samples to cover everything, and in this case, repeated indices isnt an edge case for 99.9% of solutions

Today's contest had way stronger samples than usual (and imo way



stronger than should be,
→ [Reply](#)

2 years ago, [hide](#) # | ☆

← Rev. 2 ▲ 0 ▼

Please, someone, i beg you, tell me why this C++ solution fails Problem D test 3 [241505744](#). The same exact solution works in python. There were size issues in the beginning but i addresses them. Now I just don't know what it could be and i've been trying to fix this for over 3 hours. (the python was almost correct more than 3 hours ago, but after realizing the potential size of the accumulated array can be $>>>> 10^{18}$, i made a small adjustment and it worked). But the c++ just won't and i'm just super frustrated... please :(



yael97

I'll just say briefly, there is a "recursive" class that holds the old array before multiplying (type 2 operation) by a factor, and it also holds the additions (type 1 operatoin) that happened since the last multiplication.

→ [Reply](#)



arihant_j

2 years ago, [hide](#) # | ☆

▲ 0 ▼

[241515808](#) please tell why is it giving run time error

→ [Reply](#)



FahimR

2 years ago, [hide](#) # | ☆

▲ 0 ▼

That was an amazing contest with some cool problems like C and E

→ [Reply](#)



md_nihal

2 years ago, [hide](#) # | ☆

▲ 0 ▼

Can anyone explain why calculating gcd of series of numbers take $n + \log n$ and not $n \log n$ time

Although i solved ABC but i wasn't aware of this

→ [Reply](#)



omeganot

2 years ago, [hide](#) # ^ | ☆

← Rev. 2 ▲ +11 ▼

If $\gcd(a, b) = a$ or $\gcd(a, b) = b$, the gcd algorithm will be $O(1)$. Furthermore, when we do cumulative gcd of all the numbers, our current gcd can only decrease $O(\log n)$ times (if it doesn't decrease, we have the case previously mentioned which is constant), which is why the check function runs in $O(n + \log n)$.

→ [Reply](#)

2 years ago, [hide](#) # | ☆

▲ +11 ▼

Problem C but with better time complexity. Haven't proved it mathematically but I think time complexity is $n \cdot \log(n)$. Uses the fact that if 1, 2, 4, 8, ... are factors of $2^k \mid n$ then in worst case we need to check no more than $2^k + 2^{k-1} + 2^{k-2} \dots$ which is 2^n numbers. And assuming gcds check are $\log(n)$ hence is $n \cdot \log(n)$. But math gets complicated when n is not pure power of any prime. So ig I will leave it for someone else.



Proelectro444

<https://codeforces.com/contest/1920/submission/241529452>

→ [Reply](#)



r57shell

2 years ago, [hide](#) # ^ | ☆

← Rev. 2 ▲ 0 ▼

Complexity analysis

→ [Reply](#)



Proelectro444

2 years ago, [hide](#) <#> [^](#) | [☆](#)

▲ 0 ▼

Thanks For the analysis <3

→ [Reply](#)

coderdhanraj

2 years ago, [hide](#) <#> | [☆](#)

▲ 0 ▼

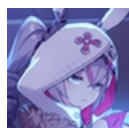
Damn, I didn't see the constraints for [1920B - Summation Game](#) and solved by considering negative numbers as well XD, [submission](#).→ [Reply](#)

dsquare

2 years ago, [hide](#) <#> | [☆](#)

▲ 0 ▼

First approach of problem D is really new learning to me. Thanks.

→ [Reply](#)

Liuxizai

2 years ago, [hide](#) <#> | [☆](#)

▲ 0 ▼

The editorial for F2 says that the time complexity of parallel binary search solution is $O((nm \cdot \alpha(nm) + q) \cdot (n + m))$. Here the $\alpha(nm)$ should from path compression in dsu, but I think we cannot use path compression since we need undo the operations, or can we?

→ [Reply](#)

jeroenodb

2 years ago, [hide](#) <#> [^](#) | [☆](#)

▲ +21 ▼

My parallel binary search solution just uses a regular DSU. You do $O(\log(n + m))$ rounds of incrementally adding all potential edges between neighbours at the right times. At a certain point in this sweep you do one connectivity query for each of the queries, no need for undoing: [241434298](#) The only thing that does not quite achieve this time complexity is some sorting I call inside the parallel binary search, but it can of course easily be fixed.

→ [Reply](#)

Liuxizai

2 years ago, [hide](#) <#> [^](#) | [☆](#)

▲ 0 ▼

That makes sense. Thanks very much.

→ [Reply](#)2 years ago, [hide](#) <#> | [☆](#)

▲ +8 ▼

Problem C

For people who didnt get why taking the gcd of difference of nos at all those positions works (found it a pretty cool idea):

Consider all the first nos from all partitions as:

$$a_1, a_2, a_3, a_4, \dots, a_k$$

Now if they have a difference which is multiple of a common number G, they can be written as: **(say)**

$$a_1, a_1+3G, a_1-4G, a_1+103G, \dots, a_1+7G$$

Now for $m = G$ you get the same remainder for this sequence: $a_1 \% G$

Now for the same no to appear at all such positions in all partitions, the required m will be the **gcd** of differences at all such positions.

I hope it makes sense now!

→ [Reply](#)

ustaad_bismillah



arihant_j

2 years ago, [hide](#) <#> | [☆](#)[▲](#) 0 [▼](#)

in problem c can someone tell that cant we just find all prime number lenghts(k) satsifying the condition (i.e $m \geq 2$) and then like we do in sieve find all the multiples of those lenghts and that will be the ans,TC of it will be $n \log n$ but idk why this approach [241560669](#) is failing

→ [Reply](#)

robostac

2 years ago, [hide](#) <#> [^](#) | [☆](#)[▲](#) 0 [▼](#)

[1,1,2,2,1,1,2,2] can't be split on any primes, but does split on $k = 4$.

→ [Reply](#)

Oleja123

2 years ago, [hide](#) <#> | [☆](#)← Rev. 2 [▲](#) 0 [▼](#)

<https://codeforces.com/contest/1920/submission/241572267>

Can someone explain why this solution of task D gets accepted on visual c++ 2017, and gets a time limit on g++20?

→ [Reply](#)

PleaseDoNotLie

2 years ago, [hide](#) <#> | [☆](#)[▲](#) 0 [▼](#)

Hey! can someone please help me out with how to calculate the time complexity for Problem C. My doubt is why is not the code in the solution given not throw TLE, The code looks like a $O(n^2)$ solution

→ [Reply](#)

omeganot

2 years ago, [hide](#) <#> [^](#) | [☆](#)[▲](#) +8 [▼](#)

When we check some divisor k , it will take $O(n + \log n)$ time since we are taking cumulative gcd of $O(n)$ numbers. The reason why the code does not take $O(n^2)$ is because the number of divisors x can have if $1 \leq x \leq 2 \cdot 10^5$ is pretty small, around 160. So the actual time complexity is closer to $O(n \cdot \text{maxdivisors})$.

→ [Reply](#)2 years ago, [hide](#) <#> | [☆](#)[▲](#) 0 [▼](#)

Had a doubt regarding problem D, would appreciate any help I could get for the same!

In the editorial I can see that they have used $2e18$ as the upper limit, but since the max length index is $1e18$, having $(1e18+5)$ would work as well right?

I did something similar during the (virtual) contest, but initially gave $1e18+5$ as the upper limit, which failed. then I gave $2e18+5$ as the upper limit, and voila! it passed! Funny thing is that the test case it failed on didn't fail on my local with $1e18+5$ limit!



harshits2808

Now I know that happens alot that a solution would fail on one IDE and pass on another, but I was unable to figure out why that happened in this case, any pointers would be highly appreciated!

Failed solution — <https://codeforces.com/contest/1920/submission/241579138>

Accepted solution — <https://codeforces.com/contest/1920/submission/241583879>

(PS : I know my both my solutions should have given TLE for a particular edge case, but I guess CF problem setters didn't think someone would be as stupid as me :P)

→ [Reply](#)

2 years ago, [hide](#) <#> | ☆

▲ 0 ▼

Can someone tell my why my Python code is getting a TLE for C. The logic seems identical to the C++ answer

```
from math import gcd
cases = int(input())

for _ in range(cases):
    n = int(input())
    nums = [int(x) for x in input().split()]

    ans = 0
    for k in range(1, n + 1):
        if n % k == 0:
            g = 0
            i = 0

            while i + k < n:
                g = gcd(g, abs(nums[i + k] - nums[i]))
                i += 1

            if g != 1:
                ans += 1

    print(ans)
```

→ [Reply](#)

jvs1992



fisk

2 years ago, [hide](#) <#> ^ | ☆

▲ +1 ▼

Python 3 is too slow. Use PyPy3 to submit your code.

[Your accepted solution.](#)

→ [Reply](#)

Adarsh_iitkgp

2 years ago, [hide](#) <#> | ☆

▲ +1 ▼

<https://codeforces.com/contest/1920/submission/241625518> Can anyone tell what is the problem with this submission? It is giving correct answer in my PC. But i guess while checking overflow there is a problem.

→ [Reply](#)

r57shell

2 years ago, [hide](#) <#> | ☆

← Rev. 2 ▲ +11 ▼

I have solution for [1920F2 - Smooth Sailing \(Hard Version\)](#) in $O(nm \cdot \alpha(nm) + q)$.

[Solution](#)

Here is the code [241633554](#)

→ [Reply](#)

r57shell

2 years ago, [hide](#) <#> ^ | ☆

▲ +8 ▼

Here is parity version: [241636798](#)

→ [Reply](#)

tim8710

2 years ago, [hide](#) <#> | ☆

← Rev. 3 ▲ 0 ▼

How does the editorial for problem F2 account for a case like this?

<https://imgur.com/a/W2gRkVa>

→ [Reply](#)



r57shell

2 years ago, [hide](#) <#> [^](#) | [☆](#)

▲ 0 ▼

Round trip is a loop. We consider only loops. In vertical segment you have on pic, you would have to go up and down, or down and up, thus there is 3 intersections of round trip on the pic (if it's cycle).

→ [Reply](#)2 years ago, [hide](#) <#> [^](#) | [☆](#)

← Rev. 2

▲ 0 ▼



timg8710

Thank you for your reply. I'm still confused about how there are 3 intersections to the right of the island — aren't we just counting how many blocks cross the dotted line? Additionally, how are we keeping track of if a component forms a round trip (or polygon) ?

→ [Reply](#)

r57shell

2 years ago, [hide](#) <#> [^](#) | [☆](#)

▲ 0 ▼

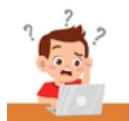
Long explanation→ [Reply](#)

limithym

2 years ago, [hide](#) <#> [^](#) | [☆](#)

▲ 0 ▼

very helpful!

→ [Reply](#)

yogesh_2249103

2 years ago, [hide](#) <#> | [☆](#)

▲ 0 ▼

in Problem C, if m is gcd of $m_1, m_2, \dots$ then all the divisor of m will also satisfy the condition of partitioning the array, then why we have only added 1 to our answer, we should add the count of divisor of m . If I am wrong please correct me?

→ [Reply](#)

timg8710

2 years ago, [hide](#) <#> [^](#) | [☆](#)

▲ 0 ▼

All the divisors of m also satisfy the condition, but that is only for a particular k . We add one point for each k for which there exists $m \geq 2$.

→ [Reply](#)2 years ago, [hide](#) <#> | [☆](#)

▲ +16 ▼

I have another solution to F2 in $O(nm \cdot \min(n, m))$ that doesn't use small to large or LCA.

My solution follows the editorial up to drawing the imaginary line. Then, I process all of the edges except those that cross the line in order of decreasing d .



avnithv

After adding each edge, create a graph where the components in the DSU are nodes. For each edge that crosses the line, add an edge between the corresponding nodes. If there is a cycle of odd length in this graph, then that cycle and all nodes connected to it form a round trip (basically the same observation as in the editorial). We can now merge all of these nodes and make a note that this component is a round trip in the DSU.

If we choose our line correctly (either horizontally or vertically), this solution runs in $O(nm \cdot \min(n, m))$. My code passes in 580 ms.

→ [Reply](#)

huikang

2 years ago, [hide](#) <#> | [☆](#)

▲ 0 ▼

The LaTeX rendered like this, initially I thought it was something nobody bothered to fix.



[Hint 1](#)
[Hint 2](#)
[Hint 3](#)
[Solution](#)

Let's first solve the problem where we are given some string s and must count the number of good substrings. To do this we use the technique of counting contributions. For every 1 in s , we find the number of good substrings containing that 1 . Consider the following example:

$$\begin{array}{ccccccc} 00001 & 0001 & 00000000 & 1 & 0001 & 000 & \\ d_1 & d_2 & & d_3 & d_4 & d_5 & \end{array}$$

Recently I found out that it is a Chrome issue — <https://affinemess.quora.com/A-brief-PSA-about-reading-math-on-Quora>

→ [Reply](#)



rutuparn

2 years ago, [hide](#) # | ☆

▲ 0 ▼

why is time complexity of problem C $\{n + \log(n)\} * \max.\text{divisors}(n)$ and not $n * \log(n) * \max.\text{divisors}(n)$?

→ [Reply](#)



Coder_GEAR5

18 months ago, [hide](#) # | ☆

▲ 0 ▼

Found an alternate approach for C. Based on the observation that 'm' would always be a prime number! Here is the java code:

```

import java.io.*;
import java.util.*;

public class Main {
    static int M = 1000000007;

    public static void main(String[] args) {
        Scanner ab = new Scanner(System.in);
        int maxN = 100000;
        int[] prime = seive(maxN);

        int T = ab.nextInt();
        for(int test = 1; test <= T; test++) {
            int N = ab.nextInt();
            int[] a = new int[N];
            for(int i=0; i<N; i++) a[i] = ab.nextInt();

            int ans = 0;
            for(int len=1; len*len <= N; len++) {
                if(N % len == 0) {
                    for(int p: prime) {
                        // if(p > len) break;
                        if(check(a, len, p)) {
                            // System.out.println(p + " " + len);
                            ++ans;
                            break;
                        }
                    }
                }
                if(len * len != N) {
                    for(int p: prime) {
                        // if(p > N/len) break;
                        if(check(a, N/len, p)) {
                            // System.out.println(p + " "
                            // + N/len);
                            ++ans;
                            break;
                        }
                    }
                }
            }
        }
    }
}

```



```

        System.out.println(ans);
    }
}

static boolean check(int[] a, int len, int m) {
    for(int i=0; i<a.length - len; i++) {
        if(a[i] % m != a[i+len] % m) return false;
    }
    return true;
}

static int[] seive(int N) {
    boolean[] isPrime = new boolean[N + 1];
    Arrays.fill(isPrime, true); // Assume all numbers are
prime
    isPrime[0] = isPrime[1] = false; // 0 and 1 are not
prime numbers

    for (int i = 2; i * i <= N; i++) {
        if (isPrime[i]) {
            for (int j = i * i; j <= N; j += i) {
                isPrime[j] = false; // Mark multiples of i
as not prime
            }
        }
    }

    List<Integer> primes = new ArrayList<>();
    for (int i = 2; i <= N; i++) {
        if (isPrime[i]) {
            primes.add(i); // Add prime numbers to the list
        }
    }

    // Convert the list to an array
    int[] primeArray = new int[primes.size()];
    for (int i = 0; i < primes.size(); i++) {
        primeArray[i] = primes.get(i);
    }

    return primeArray;
}
}
→ Reply

```

